High-Level Design (HLD)

1. System Overview

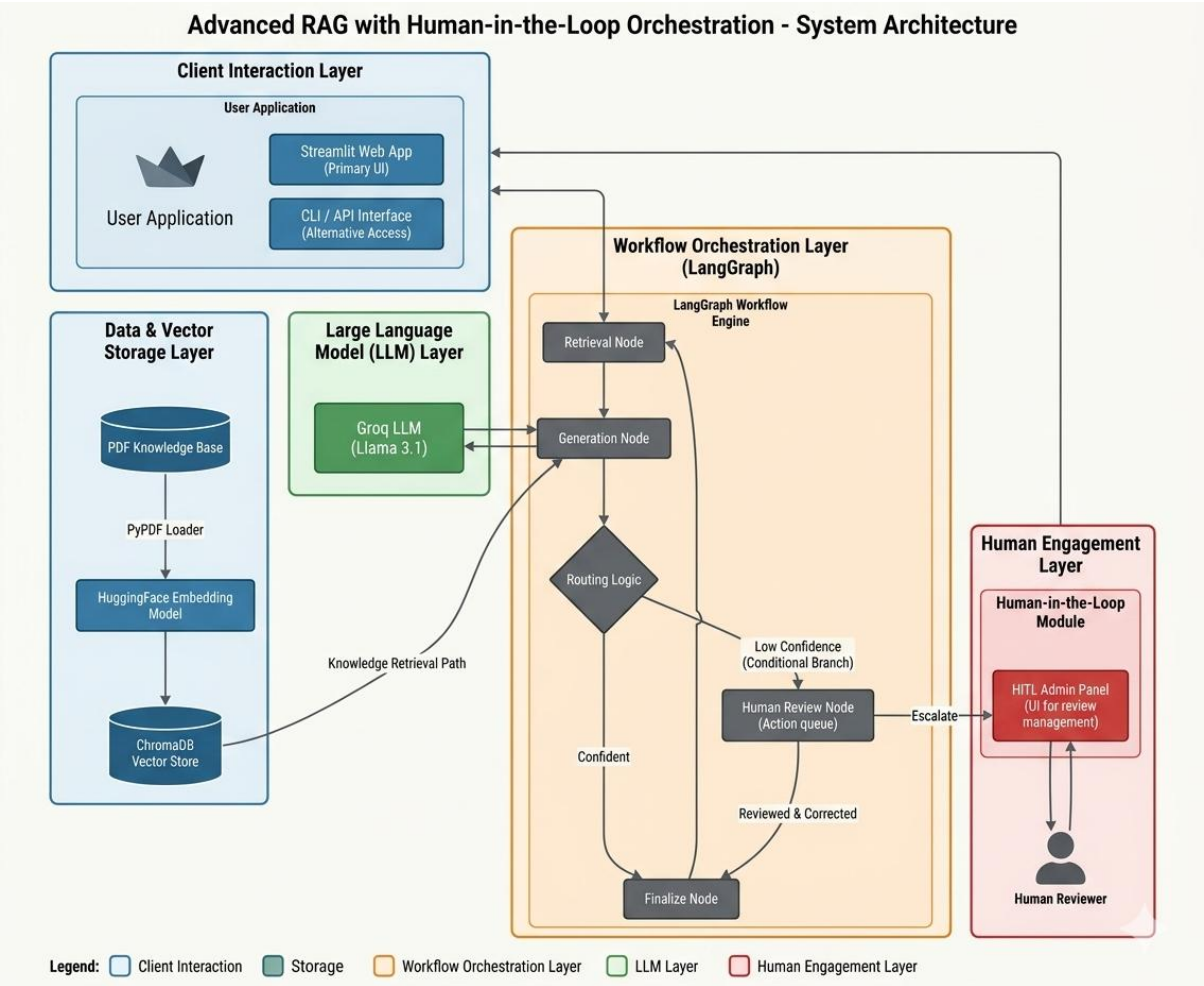
ProblemDefinition:

Traditional customer support relies on static FAQ pages or keyword search, which often fails to retrieve precise answers. Large language models alone can hallucinate or lack domain-specific knowledge. A RAG system retrieves relevant information from an internal knowledge base and generates accurate, context-grounded responses.

Scope:

This project builds a **Retrieval-Augmented Generation (RAG) Customer Support Assistant** that ingests a PDF knowledge base, indexes it in a vector database, answers user queries using a graph-based workflow, and escalates uncertain cases to a human reviewer via a Human-in-the-Loop (HITL) mechanism.

2. Architecture Diagram



3. Component Description

Component	Description
Document Loader	Uses PyPDFLoader to read PDF files and extract text.
Chunking Strategy	RecursiveCharacterTextSplitter splits text into overlapping chunks (size=500, overlap=100) to preserve context while keeping chunks manageable.
Embedding Model	all-MiniLM-L6-v2 from HuggingFace (sentence-transformers) converts text chunks into 384-dimensional vector embeddings.
Vector Store	**ChromaDB** (open-source, local) stores embeddings and supports similarity search.
Retriever	ChromaDB's similarity search with score returns the top 4 most relevant chunks for a query.
LLM	**Groq Cloud** with llama-3.1-8b-instant model generates answers based on retrieved context.
Graph Workflow Engine	**LangGraph** orchestrates the multi-step flow: retrieval → generation → conditional routing → final answer or HITL.
Routing Layer	Conditional edge should_escalate evaluates confidence score and routes to human review if below threshold.
HITL Module	Streamlit UI displays draft answer, allows human reviewer to edit, approve, or reject. LangGraph's interrupt_before pauses execution until human input is provided.

4. Data Flow

From PDF → Answer:

1. PDF loaded → split into chunks → embeddings created → stored in ChromaDB.
2. User submits query → LangGraph starts.
3. Retrieval Node fetches top-k chunks from ChromaDB and calculates confidence.
4. Generation Node sends context + query to LLM, returns draft answer.
5. Router checks confidence:
    - **High confidence** → Finalize Node delivers answer.
    - **Low confidence** → Human Review Node pauses graph, shows draft in HITL UI.
6. Human edits/approves → graph resumes → Finalize Node returns the (possibly edited) answer.
7. User sees final answer with source citations and confidence meter.

5. Technology Choices

- **Why ChromaDB?** Lightweight, open-source, Python-native, easy to embed without external servers. Perfect for prototyping and local deployment.
- **Why LangGraph?** Provides explicit state machines, native interrupt/resume capabilities essential for HITL, and clear workflow visualization.
- **LLM Choice:** Groq's llama-3.1-8b-instant offers free tier, fast inference, and good instruction-following for grounded Q&A.
- **Additional Tools:** Streamlit for rapid UI building; HuggingFace embeddings for offline, cost-free vectorization.

6. Scalability Considerations

- **Large Documents:** Can be scaled by switching to a persistent, distributed vector store (Pinecone, Weaviate) and using a serverless embedding API.
- **Increasing Query Load:** LangGraph can be deployed as a FastAPI service behind a load balancer; ChromaDB can be run in client-server mode.
- **Latency:** Embedding generation and LLM calls are the bottlenecks. Using Groq's cloud reduces LLM latency. For heavy ingestion, parallel processing can be employed.